CloudWatchX

Multi-Region AWS Infrastructure Monitoring Dashboard

Project Guide • Setup • Deployment • Troubleshooting • Security

Tagline: One Dashboard. Every AWS Region. Complete Visibility.

Built with Python, Flask, Boto3, Docker, Amazon EC2, HTML, CSS, and JavaScript.

Tagline: One Dashboard. Every AWS Region. Complete Visibility.

CloudWatchX is a self-hosted, Dockerized dashboard built with Python, Flask, Boto3, HTML, CSS, and JavaScript. It provides a centralized view of selected AWS resources across enabled regions.

1. Project overview

Main capabilities

• Discovers AWS regions enabled for the account.
• Displays EC2 instances and best-effort recent EC2 CPU utilization metrics.
• Inventories VPCs, subnets, security groups, route tables, internet gateways, NAT gateways, and EBS volumes.
• Lists Auto Scaling groups, RDS databases, Lambda functions, Elastic Load Balancing v2 load balancers, CloudWatch alarms, DynamoDB tables, ECS/EKS clusters, S3 buckets, and selected IAM resources.
• Provides a web interface with search, region filtering, service tables, and a refresh option.
• Runs in a Docker container on an EC2 instance.

Important: CloudWatchX is a personal portfolio project, not an official AWS product. It is an inventory/monitoring dashboard for the services implemented in its code; it does not cover every AWS service. Some metrics may be absent if no recent CloudWatch data exists or permissions are missing.

2. Architecture

```
Windows PC / Browser
        |
        | SSH local port forwarding (port 5000)
        v
Amazon EC2 (Amazon Linux 2023)
        |
        v
Docker container: cloudwatchx
        |
        v
Flask web application + Boto3
        |
        v
AWS APIs in enabled regions + CloudWatch metrics
```

The dashboard uses the EC2 instance's IAM role / AWS default credential chain. **Do not put long-lived AWS access keys in the script or source code.**

3. Prerequisites

• An AWS account and a running EC2 instance (Amazon Linux 2023 recommended).
• SSH access to that instance.
• A `.pem` private key that matches the EC2 instance's key pair.
• Docker installed on the instance (the setup script attempts to install and start it if needed).
• An EC2 instance role with the read permissions needed for the services you want to display.
• Windows PowerShell or Windows Terminal.

The script inventories resources and does not intentionally create AWS infrastructure. AWS usage charges can still apply to the EC2 instance, storage, or other resources you already run.

4. Download the code

Download the setup script:

[Download cloudwatchx_start_to_end.sh](cloudwatchx_start_to_end.sh)

The script generates the Flask application, HTML template, Dockerfile, and requirements file, then builds and starts the Docker container.

5. Upload the script from Windows PowerShell

Replace the example paths/IP if yours differ. Based on the setup discussed, the key file is `C:\Users\suren\Downloads\surendra_key_first.pem` and the EC2 public IP is `3.110.54.125`.

```
scp -i "C:\Users\suren\Downloads\surendra_key_first.pem" "$HOME\Downloads\cloudwatchx_start_to_end.sh" ec2-user@3.110.54.1
```

If the script is saved somewhere else, change the script path in the command to its actual location.

6. Connect to EC2 and run the setup

```
ssh -i "C:\Users\suren\Downloads\surendra_key_first.pem" ec2-user@3.110.54.125
```

Once connected, run:

```
chmod +x ~/cloudwatchx_start_to_end.sh
~/cloudwatchx_start_to_end.sh
```

The script creates the project under `~/cloudwatchx`, builds the Docker image `cloudwatchx:latest`, and runs a container named `cloudwatchx`. The app is bound to the EC2 instance's loopback interface on port 5000 for safer access.

Useful verification commands:

```
sudo docker ps
sudo docker logs --tail 100 cloudwatchx
curl http://127.0.0.1:5000/api/health
```

The health endpoint should return a small JSON response if the application is running.

7. Open the dashboard securely from Windows

Open a **second** PowerShell window on your Windows PC and run:

```
ssh -i "C:\Users\suren\Downloads\surendra_key_first.pem" -L 5000:127.0.0.1:5000 ec2-user@3.110.54.125
```

Keep this terminal open. Then open this address in your browser:

```
http://127.0.0.1:5000
```

The SSH tunnel forwards your Windows computer's port 5000 to the app on EC2. Do not expose port 5000 publicly in the EC2 security group. SSH (TCP 22) should be allowed only from your current public IP where possible.

8. IAM permissions

CloudWatchX can only show information that the EC2 instance role is allowed to read. Attach an appropriately scoped read-only policy to the EC2 instance role, then refine it to the minimum required actions for your account and services. An AWS-managed read-only policy can be useful for a learning/demo account, but it is broad; follow least privilege for production.

Do not store AWS access keys in `app.py`, shell scripts, Dockerfiles, or GitHub. Use an EC2 instance profile/role.

If some service sections show errors or no data: 1. Check the instance role and its permissions. 2. Confirm the service is available/enabled in the region. 3. Check the container logs. 4. Remember that empty results may simply mean the account has no resources of that type.

9. Common troubleshooting

SSH says the key file cannot be found

Check the exact file path:

```
Test-Path "C:\Users\suren\Downloads\surendra_key_first.pem"
```

SSH connection closes or times out

• Confirm EC2 is running and the public IP is current.

• Confirm the security group permits TCP 22 from your current public IP.

• Confirm you are using the correct key and username (`ec2-user` for Amazon Linux).

Port 5000 is already in use on Windows

Use a different local port, for example 5001:

```
ssh -i "C:\Users\suren\Downloads\surendra_key_first.pem" -L 5001:127.0.0.1:5000 ec2-user@3.110.54.125
```

Then open `http://127.0.0.1:5001`.

Container is stopped

On EC2:

```
sudo docker start cloudwatchx
sudo docker logs --tail 100 cloudwatchx
```

App responds locally but not in the Windows browser

Confirm the SSH tunnel is still running and use the matching local port in the browser. The app is intentionally bound to `127.0.0.1` on EC2, so opening the EC2 public IP on port 5000 is not expected to work.

10. Updating or restarting

Restart the current container:

```
sudo docker restart cloudwatchx
```

Inspect logs:

```
sudo docker logs -f cloudwatchx
```

Rerunning the setup script backs up the existing `~/cloudwatchx` directory before generating the app again and replaces the container named `cloudwatchx`. Review any local edits before rerunning it.

11. Suggested GitHub repository

Repository name: `cloudwatchx-aws-monitoring-dashboard`

Suggested description:

A Dockerized, Flask-based multi-region AWS resource monitoring dashboard using Boto3, with EC2 metrics, networking inventory, CloudWatch alarms, and a responsive web UI.

Suggested resume bullet:

Built a Dockerized multi-region AWS infrastructure dashboard using Python, Flask, and Boto3 to centralize visibility into EC2 instances, CPU metrics, networking resources, storage, serverless services, databases, load balancers, and CloudWatch alarms.

12. Future enhancements

• Add pagination and per-service refresh indicators.
• Add time-series charts for CloudWatch metrics.
• Add alert summaries and configurable thresholds.
• Add authentication and authorization before sharing the dashboard with others.
• Add automated tests, structured logging, and CI/CD with Jenkins.
• Add Terraform or CloudFormation for reproducible infrastructure deployment.
• Add Prometheus/Grafana only if you specifically need application/host metrics beyond the AWS inventory.

13. Security checklist

• Keep port 5000 private; access through SSH tunneling.
• Restrict SSH inbound access to your own IP.
• Use an EC2 IAM role instead of static access keys.
• Prefer least-privilege IAM permissions.
• Never commit private keys, AWS credentials, or sensitive account information to GitHub.
• Stop or terminate lab resources you no longer need to control costs.